HW 3

ENGR 21, Fall 2026.

September 24, 2026

Due Date	Thu, Sep 24
Turn in link	Gradescope code and PDF
URL	`emadmasroor.github.io/E21-F26/Homework/HW3`

What to turn in

For this assignment, you will turn in a PDF file as well as multiple code files. Each entry in the following table should be its own `.py` file.

Problem	Format
1	PDF
2	Code
3.1	Code
3.2	Code
3.3	Code
4.1	Code
4.2	Code
4.3	Code

1 Accuracy and Precision of timing the Circuit Playground Express

When we want to use the Circuit Playground Express to execute a task once every `x` seconds, where `x` is a small number, a typical approach is to write a `for` or `while` loop that contains a statement of the form `time.sleep(x)`.

For example, the following code

```
from adafruit_circuitplayground.express import cpx
import time
for j in range(5):
    cpx.pixels[j] = (50,0,0)
    time.sleep(0.5)
```

will light up a NeoPixel approximately once every 0.5 seconds.

In this problem, you will attempt to measure how close to `x` the Circuit Playground Express can get when the delay time is made smaller and smaller. You will make use of the following script, which runs a loop for `N` steps with an attempted time delay of `x` and prints out the result. The loop doesn't actually execute any code other than `sleep`ing for `x` seconds (line 14) and recording the time (line 15).

> **Lighting up the lights one by one**
>
> If you'd like to see this visually, you can substitute the following three lines in place of line 14:
>
> ```
> cpx.pixels[j % 10] = (0,0,50)
> time.sleep(delay)
> cpx.pixels[j % 10] = (0,0,0)
> ```
>
> However, this will slow down your board by a lot, so don't use this for your actual results.

```
import time
from adafruit_circuitplayground.express import cpx
from statisticalfunctions import stdev
def mean(list): return sum(list)/len(list)

# Change these parameters
delay = 0.001
N = 200


times = [0] * N
intervals = [0] * (N-1)
# Start timing
start_time = time.monotonic()
for j in range(N):
    time.sleep(delay)
    times[j] = time.monotonic() - start_time
# Done timing

for j in range(N-1):
    intervals[j] = times[j+1]-times[j]

# Uncomment the following line to see all the intervals
# [print(x) for x in intervals]
print(f"Number of iterations N = {N}")
print(f"of target duration = \t {delay} seconds")
print(f"Mean interval size \t {mean(intervals):.6f} seconds")
print(f"which is off {delay} by \t {abs(mean(intervals)-delay)/delay * 100:.3f}%")
print(f"(Max-Min)/2 = \t \t {(max(intervals)-min(intervals))/2:.6f} seconds")
print(f"Largest interval : \t {max(intervals)} seconds")
```

```
print(f"Smallest interval : \t {min(intervals)} seconds")
```

Run the code in `HW3Timing.py` on your Circuit Playground Express six times, using the following values of `N` and `delay`:

1. `N = 20`, `delay = 0.1`
2. `N = 20`, `delay = 0.01`
3. `N = 20`, `delay = 0.001`
4. `N = 200`, `delay = 0.1`
5. `N = 200`, `delay = 0.01`
6. `N = 200`, `delay = 0.001`

> **i** Example output
>
> When the instructor ran `HW3Timing.py` with `N = 200` and `delay = 0.01`, the following output was printed to the shell:
>
> ```
> Number of iterations N =    200
> of target duration =        0.01 seconds
> Mean interval size          0.010076 seconds
> which is off 0.01 by        0.763%
> (Max-Min)/2 =               0.001281 seconds
> Largest interval :          0.01135254 seconds
> Smallest interval :         0.008789064 seconds
> ```

Record the results of your experiments in the following table. One entry has been filled out from the example output above; you should replace this, too, with your own values. You will use three numbers from the output of `HW3Timing.py` to populate this table:

1. The mean interval size,
2. The 'half-spread', which `HW3Timing.py` does for you by computing `(max(intervals)-min(intervals))/2`, and
3. The percent relative error.

Items 1 and 2 are combined in a result of the form $a \pm b$, where a is the mean value and b is the 'half-spread' reported by `HW3Timing.py`. Item 3 is recorded in a separate row underneath.

Delay	$N = 20$	$N = 200$
0.1		
% Error		
0.01	0.010076 ± 0.00128	
% Error	0.763%	
0.001		
% Error		

Tasks

1. Fill in the above table, and
2. Comment on how accurately and how precisely the Circuit Playground Express is able to execute code at specified intervals, when the specified interval is `0.1` seconds, `0.01` seconds, and `0.001` seconds. How does this change when more intervals are carried out?

2 Collecting accelerometer data on the Circuit Playground Express

In class, we collected accelerometer data by `append`-ing each new reading to a list. This increases the size of the list each time a new data point is collected. In practice, programmers prefer to *pre-allocate* lists by deciding ahead of time that the list needs `N` elements.

Approach	Initialize List	Populate with entries
Append	`list1 = []`	`for i in range(10): list1.append(i)`
Pre-allocate	`list2 = [0] * 10`	`for i in range(10): list1[i] = i`

The accelerometer data-collection script below currently uses the first approach and appends data to an empty list. Re-write this script to **pre-allocate** the list instead of appending to an empty one.

If your dataset ends up having less than 100 data points, you must **trim** the list by removing all the zeros. To see how to do this, here are two hints:

- Read the documentation for lists to see how to remove elements
- Note that the statement `4 in [1,2,4,5]` is a Boolean that yields `True`, and `6 in [1,2,4,5]` is `False`.

> **Requirements**
>
> - Set a delay time of `0.1` seconds.
> - It should start collecting data immediately and end when button B is pressed or `N` data points have been collected, whichever happens first.
> - Your code should print the value of each reading.
> - After button B is pressed or `N` data points have been collected, your code should report how many zeros were removed. This number should be 0 if `N` data points were collected.

```
from adafruit_circuitplayground.express import cpx
from statisticalfunctions import stdev
import time

def magnitude(a,b,c):
    return (a**2 + b**2 + c**2)**(1/2)

# Create an empty list that will store the readings
readings = []

# Time delay between measurements
delay = 0.1

# Collect data
while True:
    accel   = magnitude(cpx.acceleration.x, cpx.acceleration.y,cpx.acceleration.z)
    print(accel)
```

```
    readings.append(accel)
    count_readings += 0
    time.sleep(delay)
    if cpx.button_b:
        break


# --- You should not need to make changes after this line --- #
true_value = 9.806
mean = sum(readings)/len(readings)
spread = abs(max(readings) - min(readings))/mean * 100
relative_error = abs(mean - true_value)/true_value * 100

print(f"After {len(readings)} readings, the accelerometer reports {mean:.3f} m/s^2 with standa
print(f"The percent error relative to the true value is {relative_error:.2f}%.")
```

> **Warning**
>
> If you see an error such as
>
> `MemoryError: memory allocation failed, allocating 896 bytes`
>
> then you must reset your board by clicking the  button.

3 Loops

3.1 Executing an action once every `n` steps

Use the Circuit Playground Express to play each note from 220 Hz to 440 Hz in increments of 10 Hz (i.e., play 220, 230, 240 ,…), holding each note for 0.2 seconds.

You may wish to consult the code provided below, which repeatedly plays 440 Hz 3 times for 0.5 seconds each.

```
from adafruit_circuitplayground.express import cpx
import time
delay = 0.2 # seconds
for i in range(3):
    cpx.start_tone(440)
    time.sleep(delay)
    cpx.stop_tone()
```

Write four independent programs that accomplish this objective using the following three approaches. Combine your code into a single `.py` file. Running your file should therefore repeat the same sound pattern four times.

1. By looping over a `range()` object (see documentation here) for which you specify a step size other than the default of 1. Do **not** use indices in this approach, i.e., you should never need to use square brackets.
2. By looping over an iterable of the form `range(N)`, where `N` is the number of notes you need to play. You should use indices in this approach.
3. Using a `while` loop and not using any for loops.
4. Using a `for` loop over the iterable `range(441)`.

> ⚠ Warning
>
> Do **not** use lists anywhere in this problem.

3.2 Nested `for` loops

Download the file `pixelpatterns.py` and save it on the `CIRCUITPY` drive. To use this file, enter the following command on the Circuit Playground:

```
from pixelpatterns import *
```

You now have access to three functions: `pattern1()`, `pattern2()` and `pattern_random()`. You should try running these functions in the REPL so that you are aware of what they do.

Write a program for the Circuit Playground Express that prints the R, G, and B value of every pixel on a new line. You should run this program after running `pattern_random()` so that your program will output the correct R, G and B values for each pixel when a random pattern is used.

> **Note**
>
> You can test your program by running it after calling `pattern1()`. This way, you know that the expected values are:
>
> ```
> Pixel number 0, index number 0 has value 29
> Pixel number 0, index number 1 has value 23
> Pixel number 0, index number 2 has value 49
> Pixel number 1, index number 0 has value 11
> ...
> ```

The output of your program should look like this, covering all 30 values:

```
Pixel number 0, index number 0 has value 76
Pixel number 0, index number 1 has value 5
...
Pixel number 3, index number 2 has value 21
Pixel number 4, index number 0 has value 73
...
Pixel number 9, index number 1 has value 2
Pixel number 9, index number 2 has value 14
```

3.3 A `while` loop for decimal-to-binary conversion

Write a Python function that takes as input a string representing a decimal number and returns as output a string representing a binary number. The expected behavior of the required function is as follows:

```
>>> decimal_to_binary("1")
'1'
>>> decimal_to_binary("2")
'10'
>>> decimal_to_binary("3")
'11'
>>> decimal_to_binary("15")
'1111'
>>> decimal_to_binary("33")
'100001'
```

Put your code in a `.py` file and name your function `decimal_to_binary`. Your code should only define the function; you do not need to call it.

> **Hints**
>
> - For decimal numbers, an object of type `str` (i.e., string) can be easily converted to an object of type `int` (i.e. integer) and vice versa.

```
>>> type("3")
<class 'str'>
>>> type(3)
<class 'int'>
>>> int("3")
3
>>> str(3)
'3'
```

- Using the `+` operator on two strings concatenates them.

```
>>> '3' + '4'
'34'
>>> 3 + 4
7
```

- It is possible to reverse a list in Python. One way to do this is the following.

```
>>> a = [1,2,3,4]
  >>> b = list(reversed(a))
  >>> b
  [4, 3, 2, 1]
```

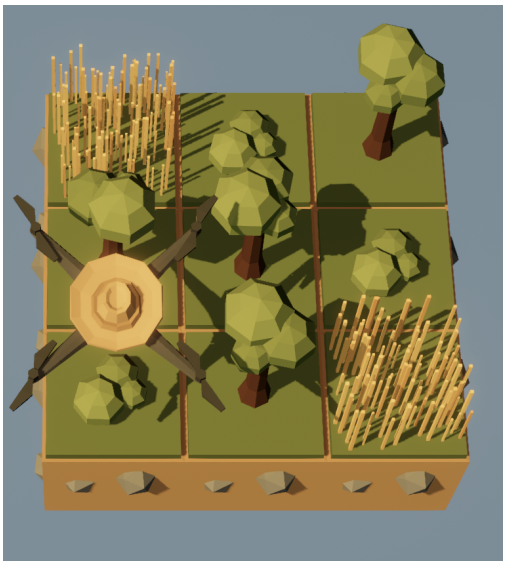
4 The Farmer Was Replaced

4.1 Planting by pattern on 3 x 3

Write a function called `plant_pattern()` that accepts as argument a list such as the following:

```
pattern1 = [[Entities.Bush,Entities.Tree,Entities.Grass],
            [Entities.Tree,Entities.Tree,Entities.Bush],
            [Entities.Grass,Entities.Bush,Entities.Tree]]
```

and uses it to plant the following pattern:



The following script illustrates the function in action.

```
def plant_pattern(pattern):
    # your code goes here.
    # no need to write a Return statement.

# An example pattern
pattern1 = [[Entities.Bush,Entities.Tree,Entities.Grass],
            [Entities.Tree,Entities.Tree,Entities.Bush],
            [Entities.Grass,Entities.Bush,Entities.Tree]]

# Start fresh
clear()

# Plant the pattern
plant_pattern(pattern1)
```

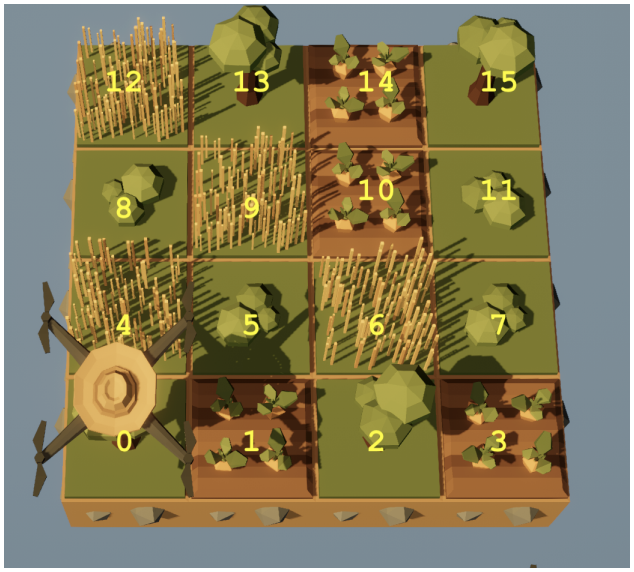
Turn in a `.py` file containing **only the function definition**. It will be tested on a different pattern during grading.

> **Note**
>
> - You must unlock the following for this assignment:
>   - Functions
>   - Lists
>   - Trees
> - You should only have a 3 x 3 world for this assignment. If you accidentally unlocked more than this, please re-start the game and get back to a world size of 3 x 3.

4.2 Planting by pattern on 4 x 4

Write a script that plants crops (once — you don't need a `while True` loop) on a 4x4 field using a pair of nested `for` loops, according to a list of locations for each plant. Each location has been given a number, and your script should check which plant goes in which location (use the `in` operator) before planting.

The numbers refer to locations on the 4x4 farm as follows:



Start from the following script.

```
# The following lists specify which crops are to be planted where.
# You may change these lists, but do not rename them.
# Note: Do **not** put any number in more than one list.

carrots = [1,14,3,10]
bushes = [5,11,7]
```

```
grass = [9,4,6,12]
trees = [13,2,15,0,8]

# Your script should clear the farm before starting.
clear()
# Your code here
```

For example, if the lists were changed to:

```
carrots = [1,14,3,12,10]
bushes = [5,11,4,7]
grass = [9,6]
trees = [13,2,15,0,8]
```

then your script should plant the pattern in Figure 1.

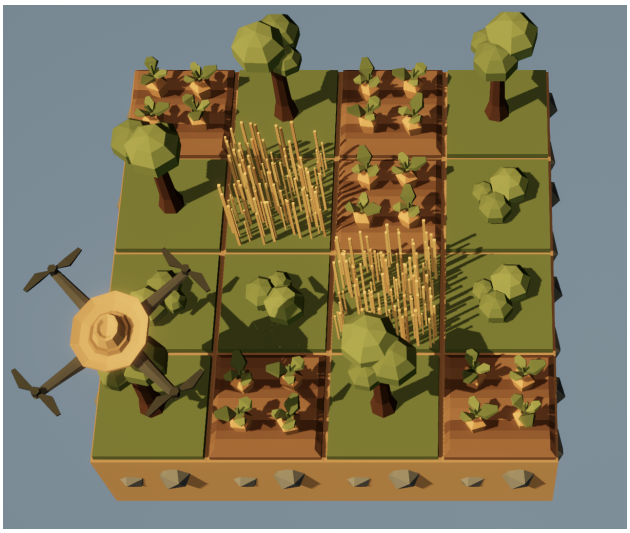


Figure 1

> **Unlocks**
>
> - Unlock anything that turns out to be necessary to complete this assignment.

To successfully complete this assignment, you should check your script by modifying the lists at the top and ensuring that the correct plants are planted at the correct locations even when changes are made to the lists.

4.3 Dictionaries in TFWR

> **Note**
>
> For this problem, please download this zip file. Unzip it and place the resulting folder `Save-for-HW3` in the folder that opens when you go to Load > Open Folder within TFWR, under the directory 'Saves'. This will allow you to start the game from a point where the features needed for this assignment have been unlocked.
> You should see `Save-for-HW3` as one of the saves that can be loaded from within the game.

In Section 4.1, we used a rather cumbersome kind of list to decide what to plant where.

For this problem, use a dictionary that will allow you to write a function that accepts as argument a simpler list of one-character 'codes' for each plant. Here, 'p' stands for pumpkin, 't' for tree, 'b' for bush and 'g' for grass.

```
pattern2 = [["c", "t", "t", "c"],
            ["t", "p", "p", "t"],
            ["t", "p", "p", "g"],
            ["c", "t", "g", "c"]]
```

Calling your function with `pattern2` as the argument should plant the crops as shown in Figure 2. Note that the entire soil is tilled; you should incorporate this into your function.

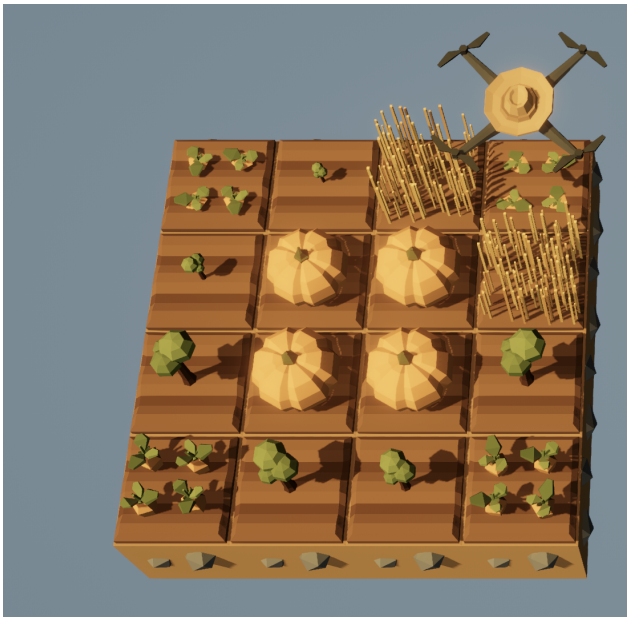


Figure 2

Turn in code with an example pattern, a dictionary definition, a function definition, and a call to your function using the example pattern, all in one `*.py` file.

```
# Your example pattern
pattern3 = [["p", "p", "t", "g"],
            ["p", "p", "g", "t"],
            ["t", "g", "p", "p"],
            ["g", "t", "p", "p"]]

# Your dictionary definition

# Your function definition
def plant_pattern(pat):
    # Your code here

# Call your function
plant_pattern(pattern3)
```